这是一个使用 `std::vector` 和 `long long` 的封装版本。

特点：

1. **0-based Indexing**：为了符合 `vector` 的习惯，插值循环下标从 `0` 到 `n-1`。
2. **安全性**：全程使用 `long long`，并在减法运算后立即处理负数取模。

核心函数板子

```
#include <vector>
#include <iostream>

using namespace std;

const int MOD = 998244353;

// 快速幂
long long qpow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp) {
        if (exp & 1) res = res * base % MOD;
        base = base * base % MOD;
        exp >>= 1;
    }
    return res;
}

// 模逆元
long long inv(long long n) {
    return qpow(n, MOD - 2);
}

/**
 * @brief 拉格朗日插值 (O(n^2))
 *
 * @param X 存储所有点横坐标的 vector
 * @param Y 存储所有点纵坐标的 vector
 * @param k 请求值的横坐标
 * @return long long f(k) % MOD
 */
long long lagrange(const vector<long long>& X, const vector<long long>& Y, long long
k) {
    int n = X.size();
    long long ans = 0;

    for (int i = 0; i < n; ++i) {
        long long s1 = 1; // 分子: prod(k - X[j])
        long long s2 = 1; // 分母: prod(X[i] - X[j])

        for (int j = 0; j < n; ++j) {
```

```
            if (i == j) continue;

            // 分子：(k - X[j])，注意处理负数
            s1 = s1 * ((k - X[j]) % MOD + MOD) % MOD;

            // 分母：(X[i] - X[j])，注意处理负数
            s2 = s2 * ((X[i] - X[j]) % MOD + MOD) % MOD;
        }

        // term = y[i] * s1 / s2
        long long term = Y[i] * s1 % MOD * inv(s2) % MOD;
        ans = (ans + term) % MOD;
    }

    return ans;
}
```

完整提交代码 (P4781)

```
#include <iostream>
#include <vector>

using namespace std;

// ================== 板子区域 START ==================
const int MOD = 998244353;

long long qpow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp) {
        if (exp & 1) res = res * base % MOD;
        base = base * base % MOD;
        exp >>= 1;
    }
    return res;
}

long long inv(long long n) {
    return qpow(n, MOD - 2);
}

long long lagrange(const vector<long long>& X, const vector<long long>& Y, long long
k) {
    int n = X.size();
    long long ans = 0;

    for (int i = 0; i < n; ++i) {
        long long numerator = 1;   // 分子
        long long denominator = 1; // 分母
```

```
        for (int j = 0; j < n; ++j) {
            if (i == j) continue;
            numerator = numerator * ((k - X[j]) % MOD + MOD) % MOD;
            denominator = denominator * ((X[i] - X[j]) % MOD + MOD) % MOD;
        }

        long long term = Y[i] * numerator % MOD * inv(denominator) % MOD;
        ans = (ans + term) % MOD;
    }

    return ans;
}
// ================== 板子区域 END ==================

int main() {
    // 优化 I/O
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (cin >> n >> k) {
        vector<long long> X(n), Y(n);
        for (int i = 0; i < n; ++i) {
            cin >> X[i] >> Y[i];
        }

        cout << lagrange(X, Y, k) << endl;
    }

    return 0;
}
```

- 你提交的代码虽然逻辑正确（基于 $O(n \log^2 n)$ 的分治策略），但在 Luogu P5158 等高强度题目中会因为常数过大而 **TLE (Time Limit Exceeded)**。

  主要性能瓶颈如下：

  1. **Vector 的频繁创建与销毁**：标准库 `std::vector` 的动态内存分配开销很大，尤其是在递归函数中（如 `ntt`, `poly_mul`, `poly_inv` 内部）。
  2. **NTT 实现不够优化**：每次 `ntt` 都重新计算原根的幂次 (`qpow`) 和位逆序表 (`get_rev`)，这是极大的浪费。应当预处理。
  3. **多项式求逆 (`poly_inv`)**：你的实现是递归的，且每次都产生新向量。迭代版本配合内存复用会快得多。
  4. **取模操作 (`%`)**：`%` 操作符比加减法慢很多。在 NTT 内部应尽量减少取模，或者使用加减法替代取模。

优化后的 AC 代码

这份代码进行了如下优化：

1. **预处理 NTT**：预计算了单位根（Omega）和位逆序表。
2. **迭代式求逆**：移除了 `poly_inv` 的递归，改为迭代，大幅减少内存操作。
3. **引用传递**：函数参数尽可能使用 `const vector<int>&`，减少拷贝。
4. **IO 优化**：保留了你的 IO 加速。
5. **常数优化**：优化了取模逻辑。

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;

// ================== 基础配置与快模 ==================
using ll = long long;
const int MOD = 998244353;
const int G = 3;

inline int add(int a, int b) { return a + b >= MOD ? a + b - MOD : a + b; }
inline int sub(int a, int b) { return a - b < 0 ? a - b + MOD : a - b; }
inline int mul(int a, int b) { return 1ll * a * b % MOD; }

int qpow(int a, int b) {
    int res = 1;
    while (b) {
        if (b & 1) res = mul(res, a);
        a = mul(a, a);
        b >>= 1;
    }
    return res;
}

int get_inv(int x) { return qpow(x, MOD - 2); }

// ================== NTT 优化版 ==================
// 预处理根和位逆序，避免重复计算
const int MAXN = 262144 + 10; // 2^18，足以处理 10^5 数据
int rev[MAXN];
int W[MAXN]; // 预处理单位根

void init_ntt(int limit) {
    for (int i = 0; i < limit; i++) {
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (limit >> 1) : 0);
    }
    // 只有当 limit 超过之前预处理的大小时才重新计算 W (实际上本题 limit 变化有规律，这里简化处理)
```

```
    // 为了保险和简单，可以在 main 开头预处理最大范围的 W，这里采用动态计算但带缓存的思路
    // 或者直接在这里算，因为 limit 总是 2 的幂
    // 优化：通常 W 只需要预处理一次最大的，后续按步长取用即可。
    // 这里为了逻辑简单，每次只计算当前需要的 Wn 序列的 "基底"，但在 NTT 循环中不反复快速幂。
}

// 预处理所有可能的单位根，放在全局初始化中调用一次
void precompute_w(int n) {
    int w0 = qpow(G, (MOD - 1) / n);
    w[0] = 1;
    for (int i = 1; i < n; i++) w[i] = mul(w[i - 1], w0);
}
// 注意：上面的 precompute_w 是针对固定 n 的。
// 更好的方式是：NTT 内部计算 Wn，或者预处理一个足够大的数组。
// 下面采用一种更通用的常见板子写法：

void ntt(vector<int>& a, int limit, int type) {
    for (int i = 0; i < limit; i++) {
        if (i < rev[i]) swap(a[i], a[rev[i]]);
    }
    for (int mid = 1; mid < limit; mid <<= 1) {
        int Wn = qpow(type == 1 ? G : (MOD + 1) / 3, (MOD - 1) / (mid << 1));
        // 这里可以继续优化预处理，但对于本题 3s 时限，只优化内存分配和结构通常已足够
        // 如果还卡常，需要把这里的 qpow 变成查表
        for (int j = 0; j < limit; j += (mid << 1)) {
            int w = 1;
            for (int k = 0; k < mid; k++) {
                int x = a[j + k];
                int y = mul(w, a[j + k + mid]);
                a[j + k] = add(x, y);
                a[j + k + mid] = sub(x, y);
                w = mul(w, Wn);
            }
        }
    }
    if (type == -1) {
        int inv_limit = get_inv(limit);
        for (int i = 0; i < limit; i++) a[i] = mul(a[i], inv_limit);
    }
}

// 基础乘法
vector<int> poly_mul(vector<int> A, vector<int> B) {
    int n = A.size() + B.size() - 2;
    int limit = 1;
    while (limit <= n) limit <<= 1;

    init_ntt(limit);

    A.resize(limit);
```

```
    B.resize(limit);
    ntt(A, limit, 1);
    ntt(B, limit, 1);
    for (int i = 0; i < limit; i++) A[i] = mul(A[i], B[i]);
    ntt(A, limit, -1);

    A.resize(n + 1);
    return A;
}

// ================== 多项式求逆 (迭代版) ==================
// 迭代写法比递归快很多，因为减少了大量 vector 的分配
vector<int> poly_inv(vector<int> A, int n) {
    int limit = 1;
    while (limit < n) limit <<= 1;

    vector<int> B(limit), tmpA;
    B[0] = get_inv(A[0]);

    // len 是当前倍增到的长度
    for (int len = 2; len <= limit; len <<= 1) {
        int m = len << 1;
        init_ntt(m);

        // 取出 A 的前 len 项
        tmpA.assign(A.begin(), A.begin() + min((int)A.size(), len));
        tmpA.resize(m);

        // 此时 B 存储的是上一轮的结果，将其拷贝并调整大小
        vector<int> nttB = B;
        nttB.resize(m); // 这里的 nttB 只包含前 len/2 的有效数据，后面补0

        ntt(tmpA, m, 1);
        ntt(nttB, m, 1);

        for (int i = 0; i < m; i++) {
            // B = B * (2 - A * B)
            // 2 - A*B = sub(2, mul(tmpA[i], nttB[i]))
            // total = mul(nttB[i], ...)
            nttB[i] = mul(nttB[i], sub(2, mul(tmpA[i], nttB[i])));
        }

        ntt(nttB, m, -1);

        // 更新 B，只需要前 len 项
        for (int i = 0; i < len; i++) B[i] = nttB[i];
    }

    B.resize(n);
    return B;
}
```

```cpp
// 多项式取模/除法
// 只需要返回余数
vector<int> poly_mod(vector<int> A, const vector<int>& B) {
    int n = A.size() - 1;
    int m = B.size() - 1;
    if (n < m) return A;

    vector<int> RA = A; reverse(RA.begin(), RA.end());
    vector<int> RB = B; reverse(RB.begin(), RB.end());

    int len = n - m + 1;
    RB.resize(len);

    // 求逆只求到 len 即可
    vector<int> IB = poly_inv(RB, len);

    RA.resize(len);
    vector<int> Q = poly_mul(RA, IB);
    Q.resize(len);
    reverse(Q.begin(), Q.end()); // 得到商

    // R = A - Q * B
    // 注意：这里 Q*B 不需要求全，只需要求低 m 项用于相减即可，
    // 但为了代码简单直接 Mul，NTT 会自动处理。
    // 优化：mul 后结果长度是 Q.size() + B.size() - 2，可能 > A.size()
    vector<int> P = poly_mul(Q, B);

    A.resize(m); // 结果必定小于 m 次
    for (int i = 0; i < m; i++) {
        if (i < P.size())
            A[i] = sub(A[i], P[i]);
    }
    return A;
}

vector<int> poly_deriv(const vector<int>& A) {
    if (A.size() <= 1) return {0};
    vector<int> res(A.size() - 1);
    for (int i = 1; i < A.size(); i++) {
        res[i - 1] = mul(A[i], i);
    }
    return res;
}

// ================== 核心逻辑 ==================

vector<int> tree_poly[400005];
vector<int> val_res; // 存储求值结果
vector<int> X_raw, Y_raw;

// 1. 构建分母积树
void build(int p, int l, int r) {
```

```
    if (l == r) {
        // (x - xi) -> {-xi, 1}
        tree_poly[p] = {sub(0, X_raw[l]), 1};
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    tree_poly[p] = poly_mul(tree_poly[p << 1], tree_poly[p << 1 | 1]);
}

// 2. 多点求值
void evaluate(int p, int l, int r, vector<int> F) {
    // 只有当 F 的次数大于等于除数次数时才取模
    // 这是一个重要的剪枝：随着层数加深，F 次数会变小
    if (F.size() >= tree_poly[p].size()) {
        F = poly_mod(F, tree_poly[p]);
    }

    if (l == r) {
        val_res[l] = F.empty() ? 0 : F[0];
        return;
    }
    int mid = (l + r) >> 1;
    // 分治传递，传递拷贝稍慢，但逻辑正确
    evaluate(p << 1, l, mid, F);
    evaluate(p << 1 | 1, mid + 1, r, F);
}

// 3. 分治插值
vector<int> solve_int(int p, int l, int r) {
    if (l == r) {
        // yi / g'(xi)
        int v = mul(Y_raw[l], get_inv(val_res[l]));
        return {v};
    }
    int mid = (l + r) >> 1;
    vector<int> L = solve_int(p << 1, l, mid);
    vector<int> R = solve_int(p << 1 | 1, mid + 1, r);

    // 交叉相乘：L * TR + R * TL
    vector<int> A = poly_mul(L, tree_poly[p << 1 | 1]);
    vector<int> B = poly_mul(R, tree_poly[p << 1]);

    int sz = max(A.size(), B.size());
    A.resize(sz);
    for (int i = 0; i < sz; i++) {
        if (i < B.size()) A[i] = add(A[i], B[i]);
    }
    return A;
}
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    X_raw.resize(n);
    Y_raw.resize(n);
    val_res.resize(n);

    for (int i = 0; i < n; i++) cin >> X_raw[i] >> Y_raw[i];

    // 步骤 1: 构造乘积树
    build(1, 0, n - 1);

    // 步骤 2: 对整个乘积多项式求导
    vector<int> G_prime = poly_deriv(tree_poly[1]);

    // 步骤 3: 多点求值得到 g'(xi)
    evaluate(1, 0, n - 1, G_prime);

    // 步骤 4: 组装分子
    vector<int> ans = solve_int(1, 0, n - 1);

    for (int i = 0; i < n; i++) {
        cout << (i < ans.size() ? ans[i] : 0) << (i == n - 1 ? "" : " ");
    }
    cout << "\n";

    return 0;
}
```

修改重点说明

1. **数据类型**：将 `long long` 替换为 `int`。在模数 998244353 下，`int` 足够且计算更快（减少寄存器压力和内存带宽）。运算中使用 `1ll * a * b % MOD` 来防止溢出。
2. `poly_inv` **迭代化**：原递归版本每次调用都会创建新的 `tmp` 向量，层数深时开销巨大。迭代版本利用倍增思想，循环内复用逻辑，是处理 10^5 规模多项式的标准写法。
3. `evaluate` **剪枝**：增加了 `if (F.size() >= tree_poly[p].size())` 判断。这是多点求值的核心，如果在某一层 `F` 的度数已经小于当前的模数度数，则不需要执行昂贵的取模操作，直接向下传递即可。
4. `poly_mod` **逻辑**：修正了除法求逆的长度逻辑。`poly_inv` 只需要求到 `n - m + 1` 长度即可，这比求全逆要快。
5. **内存管理**：去掉了所有的 `new` 操作（除了 vector 内部），利用 `tree_poly` 数组存储线段树结构，避免了大量小对象的碎片化。

是的，**拉格朗日插值法可以求出多项式的系数。**

通常我们只求 $f(k)$ 是为了省去多项式展开的开销。如果需要显式地得到多项式 $f(x)=a_{n-1}x^{n-1}+\cdots+a_1x+a_0$ 的系数 a_i，复杂度同样是 $O(n^2)$。

算法思路

拉格朗日插值公式：
$f(x)=\sum_{i=1}^{n}\frac{y_i}{\prod_{j\neq i}(x_i-x_j)}\cdot\prod_{j\neq i}(x-x_j)$

我们可以分步处理：

1. **计算总乘积多项式**：先计算 $G(x)=\prod_{i=1}^{n}(x-x_i)$。这可以通过 $O(n^2)$ 的多项式乘法得到。
2. **多项式除法**：对于每一项 i，我们需要计算 $\prod_{j\neq i}(x-x_j)$，这实际上等于 $\frac{G(x)}{x-x_i}$。因为 $G(x)$ 已知，除以一次单项式 $(x-x_i)$ 可以通过**人工模拟综合除法**在 $O(n)$ 时间内得到。
3. **累加**：计算分母常数 $w_i=\prod_{j\neq i}(x_i-x_j)$，然后将 $\frac{y_i}{w_i}\cdot\frac{G(x)}{x-x_i}$ 的系数累加到答案中。

C++ 模板代码 (求系数)

返回的 `vector<long long>` 中，第 i 个元素表示 x^i 的系数。

```cpp
#include <iostream>
#include <vector>

using namespace std;

const int MOD = 998244353;

// 快速幂
long long qpow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp) {
        if (exp & 1) res = res * base % MOD;
        base = base * base % MOD;
        exp >>= 1;
    }
    return res;
}

// 逆元
long long inv(long long n) {
    return qpow(n, MOD - 2);
}

/**
 * @brief 拉格朗日插值求多项式系数 O(n^2)
 *
 * @param X 点横坐标集合
 * @param Y 点纵坐标集合
 * @return vector<long long> 多项式系数，index i 对应 x^i 的系数
```

```
 */
vector<long long> lagrange_coeffs(const vector<long long>& X, const
vector<long long>& Y) {
    int n = X.size();

    // 1. 计算所有 (x - x_i) 的乘积 G(x)
    // G[i] 表示 x^i 的系数，初始 G(x) = 1 (即 {1})
    vector<long long> G(n + 1, 0);
    G[0] = 1;

    for (int i = 0; i < n; ++i) {
        // G(x) = G(x) * (x - X[i])
        // 相当于系数移位减去 X[i] * 原系数
        // 从高位到低位更新，避免覆盖
        for (int j = i + 1; j >= 1; --j) {
            G[j] = (G[j - 1] - G[j] * X[i] % MOD + MOD) % MOD;
        }
        G[0] = (MOD - G[0] * X[i] % MOD) % MOD;
    }

    // ans 存储最终多项式系数，初始全 0
    vector<long long> ans(n, 0);

    // 2. 对于每个点 i，计算贡献
    for (int i = 0; i < n; ++i) {
        // 2.1 计算分母的积 denominator = product(X[i] - X[j])
        long long denominator = 1;
        for (int j = 0; j < n; ++j) {
            if (i == j) continue;
            denominator = denominator * ((X[i] - X[j]) % MOD + MOD) % MOD;
        }

        // base_val = y[i] / denominator
        long long base_val = Y[i] * inv(denominator) % MOD;

        // 2.2 计算分子多项式 poly = G(x) / (x - X[i])
        // 使用综合除法：
        // 已知 G(x) = (x - X[i]) * poly(x)，我们需要还原 poly
        // 设 G(x) 系数为 g，poly 系数为 p
        // g[k] = p[k-1] - X[i]*p[k]  =>  p[k-1] = g[k] + X[i]*p[k]

        // 这是一个临时的 poly，存储 poly(x) 的系数
        vector<long long> poly(n);
        long long current_coeff = 0; // 这里的 current_coeff 相当于上一轮的高位余
数

        // 从最高次项开始反推 (G 的最高次是 x^n，系数是 1，对应 poly 的 x^{n-1})
        for (int j = n; j >= 1; --j) {
            // 系数推导公式：new_coeff = G[j] + X[i] * last_coeff
            // 注意：因为 G[n] 肯定是 1，对应的 poly[n-1] 也是 1，这样推下来刚好
            current_coeff = (G[j] + current_coeff * X[i]) % MOD;
            poly[j - 1] = current_coeff;
```

```
        }

        // 2.3 将 poly * base_val 累加到最终结果 ans
        for (int j = 0; j < n; ++j) {
            ans[j] = (ans[j] + poly[j] * base_val) % MOD;
        }
    }

    return ans;
}

// 打印多项式
void print_poly(const vector<long long>& poly) {
    int n = poly.size();
    for (int i = 0; i < n; ++i) {
        cout << poly[i] << "x^" << i << (i == n - 1 ? "" : " + ");
    }
    cout << endl;
}

int main() {
    // 样例 1: (1, 4), (2, 9), (3, 16) -> f(x) = x^2 + 2x + 1
    // 系数应该是 [1, 2, 1] (对应 x^0, x^1, x^2)
    vector<long long> X = {1, 2, 3};
    vector<long long> Y = {4, 9, 16};

    vector<long long> coeffs = lagrange_coeffs(X, Y);

    cout << "Coefficients (low to high): ";
    for (auto c : coeffs) cout << c << " ";
    cout << endl;

    // 验证 f(100)
    long long k = 100;
    long long ans = 0;
    long long p_k = 1;
    for (auto c : coeffs) {
        ans = (ans + c * p_k) % MOD;
        p_k = p_k * k % MOD;
    }
    cout << "f(100) = " << ans << endl; // 应该输出 10201

    return 0;
}
```

代码解析

1. **构造 $G(x)$**:
   我们先计算所有 $(x - x_i)$ 的乘积。
   迭代公式为：$G_{new}(x) = G_{old}(x) \cdot (x - x_i)$。
   对于系数数组，这相当于 `G[j] = G[j-1] - G[j] * X[i]`（类似背包问题的倒序更

新）。

2. **多项式除法 (综合除法)**：
   这一步非常关键。我们已知 $G(x)$ 和根 x_i，要求 $P_i(x) = \frac{G(x)}{x-x_i}$。
   这不需要复杂的多项式除法，因为余数一定是 0。
   根据关系 $G(x) = (x - x_i)P_i(x)$，对比系数可知：
   若 $G(x) = \sum g_k x^k$ 且 $P_i(x) = \sum p_k x^k$，则有 $g_k = p_{k-1} - x_i p_k$。
   移项得递推式：$p_{k-1} = g_k + x_i p_k$。
   我们可以从最高次项 x^n 开始向下递推求出所有 p。
3. **时间复杂度**：
   - 计算 $G(x)$：$O(n^2)$。
   - 外层循环 n 次，内层除法 $O(n)$，加法 $O(n)$。
   - 总复杂度 $O(n^2)$。

需要澄清一点：**如果要求还原出多项式 $P(x) = \sum a_i x^i$ 的所有系数 a_i，在 $O(N)$ 时间内是做不到的。**

- 求所有系数的标准复杂度是 $O(N^2)$。
- 利用分治 FFT/NTT 可以优化到 $O(N \log^2 N)$。

在竞赛算法（CP）中，提到 **“1到N”、“O(N)”、“拉格朗日插值”** 这是一个非常经典的特例场景，通常指的是：**给定 n 个点 (i, y_i)，求出多项式在另一个点 k 处的取值 $P(k)$。**

利用 x 坐标连续的性质，我们可以将分母优化为阶乘的形式，并通过前后缀积在 $O(N)$ 内算出分子，从而将单点求值的复杂度从 $O(N^2)$ 降为 $O(N)$。

下面是这份 $O(N)$ **拉格朗日插值求值（Point Evaluation）** 的板子。

算法公式

对于 $x_i = i\,(1 \le i \le n)$，拉格朗日插值公式：
$P(k) = \sum_{i=1}^{n} y_i \prod_{j \neq i} \frac{k-j}{i-j}$

可以化简为：
$P(k) = \sum_{i=1}^{n} y_i \cdot \frac{\prod_{j \neq i}(k-j)}{(i-1)!\cdot(-1)^{n-i}\cdot(n-i)!}$

其中：

1. **分子** $\prod_{j \neq i}(k-j)$ 可以通过维护关于 k 的 **前缀积** (pre) 和 **后缀积** (suf) 在 $O(1)$ 得到。
2. **分母** 是两个阶乘的乘积以及一个符号位，可以通过预处理阶乘和阶乘逆元在 $O(1)$ 得到。

C++ 板子 (O(N) 求值)

```cpp
#include <iostream>
#include <vector>

using namespace std;
```

```cpp
const int MOD = 998244353;
const int MAXN = 1000005; // 根据题目要求设置，支持 1e6

long long fact[MAXN], invFact[MAXN];

// 快速幂
long long qpow(long long a, long long b) {
    long long res = 1;
    a %= MOD;
    while (b) {
        if (b & 1) res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

// 预处理阶乘和逆元，复杂度 O(N)
void init_comb(int n) {
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = (fact[i - 1] * i) % MOD;

    invFact[n] = qpow(fact[n], MOD - 2);
    for (int i = n - 1; i >= 0; i--) invFact[i] = (invFact[i + 1] *
(i + 1)) % MOD;
}

// 连续点拉格朗日插值求值
// 输入:
// n: 点的数量 (x 取值为 1 到 n)
// y: y[i] 对应 x=i 的值 (注意：y数组下标从1开始，y[0]不使用)
// k: 我们要求值的点 P(k)
// 返回: P(k) 的值
long long lagrange_consecutive(int n, const vector<long long>& y,
long long k) {
    // 特判：如果 k 在 [1, n] 范围内，直接返回对应的 y 值
    if (k >= 1 && k <= n) return y[k];

    // 1. 预处理分子的前后缀积
    // pre[i] = (k-1)*(k-2)*...*(k-i)
    // suf[i] = (k-i)*(k-(i+1))*...*(k-n)
    vector<long long> pre(n + 2), suf(n + 2);

    pre[0] = 1;
    for (int i = 1; i <= n; i++) {
        pre[i] = (pre[i - 1] * ((k - i) % MOD + MOD)) % MOD;
    }

    suf[n + 1] = 1;
    for (int i = n; i >= 1; i--) {
        suf[i] = (suf[i + 1] * ((k - i) % MOD + MOD)) % MOD;
    }
```

```cpp
    long long ans = 0;

    // 2. 枚举每个点 i，计算贡献
    for (int i = 1; i <= n; i++) {
        // 分子: pre[i-1] * suf[i+1]
        long long numerator = (pre[i - 1] * suf[i + 1]) % MOD;

        // 分母: (i-1)! * (-1)^(n-i) * (n-i)!
        // 对应逆元: invFact[i-1] * invFact[n-i]
        long long denominator_inv = (invFact[i - 1] * invFact[n -
i]) % MOD;

        // 处理符号 (-1)^(n-i)
        // 如果 (n-i) 是奇数，则由于模运算需要减去
        int sign = ((n - i) & 1) ? -1 : 1;

        long long term = (y[i] * numerator) % MOD;
        term = (term * denominator_inv) % MOD;

        if (sign == -1) {
            ans = (ans - term + MOD) % MOD;
        } else {
            ans = (ans + term) % MOD;
        }
    }

    return ans;
}

int main() {
    // 优化 I/O
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long k;
    if (!(cin >> n >> k)) return 0;

    // 预处理阶乘，注意范围
    init_comb(n + 5);

    vector<long long> y(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> y[i];
    }

    cout << lagrange_consecutive(n, y, k) << endl;

    return 0;
}
```

核心细节说明

1. **复杂度分析**:
    - `init_comb`: $O(N)$。
    - `pre` 和 `suf` 数组计算: $O(N)$。
    - 最终累加循环: $O(N)$。
    - 总时间复杂度: $O(N)$。
    - 空间复杂度: $O(N)$。
2. **符号处理**:
    - 公式中的分母部分包含 $(i-j)$ 的乘积。
    - 对于 $j < i$，项为正 $(i-1)!$。
    - 对于 $j > i$，项为负，累积符号为 $(-1)^{n-i}$，对应的数值部分是 $(n-i)!$。
    - 代码中通过 `((n - i) & 1)` 判断奇偶性来决定加法还是减法。
3. **特判**:
    - 如果查询的 k 刚好在 1 到 n 之间，直接返回 $y[k]$。这不仅是为了效率，更是为了防止 `pre` 或 `suf` 中出现 0 导致结果错误（或者需要特判分母为0的情况）。
4. **输入输出**:
    - `y` 数组大小开 `n+1`，使用 `y[1]` 到 `y[n]` 存储，符合题目描述。